

MoEJointQuant: Modeling, Guarantees, and Staged Verification of Cross-Expert Post-Training Quantization

Method note and executable toy study

August 2026

Decision. Validate the uncompressed full-joint model before designing a scalable Hessian approximation. The first-stage method is an exact dense co-routing objective plus a finite discrete coordinate solver initialized from an expert-local baseline. This gives a no-worse guarantee on the calibration objective. Acceleration and specialized calibration data are separate hypotheses and must not be used to rescue a failed joint formulation.

Abstract

Existing MoE post-training quantization commonly reconstructs each expert independently. This is not the deployed objective: a routed MoE block returns a weighted sum of simultaneously active experts, so its reconstruction loss contains cross-expert terms. This note formulates the exact joint down-projection quantization problem by concatenating gate-weighted expert activations. It proves that the globally optimal joint solution cannot be worse than any independently produced solution under the same quantization lattice, and gives a practical finite coordinate-descent correction whose calibration objective is monotonically non-increasing when initialized from affinity-weighted GPTQ. A small exhaustive oracle separates modeling gain from solver gap. A five-seed synthetic Top-2 MoE study then tests whether this gain transfers to held-out block output, a nonlinear suffix, and the next routing decision. The study also exposes a calibration-distribution failure: random samples from a skewed task mixture do not cover balanced co-routing behavior. Route-balanced selection repairs much of this gap, but is treated as a second-stage result rather than part of the core claim.

Contents

1	Scope and staged research question	3
2	Exact MoE block formulation	3
3	What independent quantization discards	4
4	What can actually be guaranteed	4
5	From block error to later routing	5
6	Calibration as a separate generalization problem	6
7	Executable toy MoE experiment	7
8	Most feasible real-model verification	9
9	Deferred acceleration	9
10	Reproducibility	10
11	Conclusion	10

1 Scope and staged research question

The central question is deliberately narrow:

Does optimizing the exact aggregated MoE-block reconstruction objective produce a better quantized block than optimizing experts independently, before any Hessian compression, route fine-tuning, or end-to-end correction is introduced?

The project is divided into three gates.

1. **Model verification.** On a tiny problem, compute the exact independent and exact joint discrete optima. The joint objective must exhibit a non-trivial oracle gain.
2. **Solver verification.** On a larger toy MoE, use the full dense Hessian. A feasible joint solver must convert calibration gain into held-out block and suffix gain.
3. **Scaling.** Only after the first two gates pass should the work introduce graph grouping, slicing, sparse-plus-low-rank decomposition, or approximate inverse updates.

This ordering avoids a common attribution error: if an accelerated method fails, one cannot otherwise tell whether the joint model is wrong or the approximation destroyed its benefit.

2 Exact MoE block formulation

2.1 Routed down projection as one linear operator

Consider one MoE feed-forward block with E experts and Top- k routing. For calibration token n , let $g_{ne} \geq 0$ be the post-Top- k gate weight and let $\mathbf{a}_{ne} \in \mathbb{R}^d$ be the hidden activation immediately before expert e 's down projection $W_e \in \mathbb{R}^{d \times m}$. Inactive experts have $g_{ne} = 0$. The deployed block output is

$$\mathbf{y}_n = \sum_{e=1}^E g_{ne} \mathbf{a}_{ne}^\top W_e. \quad (1)$$

Define the gate-weighted activation blocks

$$\Phi_e = \begin{bmatrix} g_{1e} \mathbf{a}_{1e}^\top \\ \vdots \\ g_{Ne} \mathbf{a}_{Ne}^\top \end{bmatrix} \in \mathbb{R}^{N \times d}, \quad \Phi = [\Phi_1 \mid \cdots \mid \Phi_E] \in \mathbb{R}^{N \times Ed}, \quad (2)$$

and vertically stack all down projections,

$$\mathbf{W} = [W_1^\top, \dots, W_E^\top]^\top \in \mathbb{R}^{Ed \times m}. \quad (3)$$

Then the complete MoE output matrix is exactly

$$\mathbf{Y} = \Phi \mathbf{W}. \quad (4)$$

No linearization or suffix Jacobian is needed for this identity. It is exact once the expert hidden activations and router weights are fixed.

2.2 The exact joint Hessian

Let $\mathbf{Q} \in \mathcal{Q}$ be the stacked quantized weights, where $\mathcal{Q} = \mathcal{Q}_1 \times \cdots \times \mathcal{Q}_E$ is the same product of expert quantization lattices used by an independent method. The normalized block reconstruction loss is

$$\mathcal{L}_{\text{joint}}(\mathbf{Q}) = \frac{1}{N} \|\Phi(\mathbf{W} - \mathbf{Q})\|_F^2 \quad (5)$$

$$= \text{tr}\left((\mathbf{W} - \mathbf{Q})^\top \mathbf{H} (\mathbf{W} - \mathbf{Q})\right), \quad \mathbf{H} = \frac{1}{N} \Phi^\top \Phi \succeq 0. \quad (6)$$

The (e, j) expert block is

$$H_{ej} = \frac{1}{N} \sum_{n=1}^N g_{ne} g_{nj} \mathbf{a}_{ne} \mathbf{a}_{nj}^\top. \quad (7)$$

Thus Top- k routing induces expert-level sparsity: $H_{ej} = 0$ whenever e and j never co-activate in calibration. However, an observed co-routing edge generally creates a nonzero cross block.

Proposition 1 (Exact objective identity). *For every feasible quantized weight $\mathbf{Q} \in \mathcal{Q}$, equations (5) and (6) equal the empirical aggregated MoE-block output MSE. Therefore minimizing the quadratic is neither a per-expert proxy nor a route-similarity surrogate.*

Proof. Substitute $Y = \Phi \mathbf{W}$ and $\hat{Y} = \Phi \mathbf{Q}$ into $N^{-1} \|Y - \hat{Y}\|_F^2$, then use $\|A\|_F^2 = \text{tr}(A^\top A)$. $\square$

3 What independent quantization discards

Write $\Delta_e = W_e - Q_e$. Expanding (5) gives

$$\mathcal{L}_{\text{joint}} = \sum_e \text{tr}(\Delta_e^\top H_{ee} \Delta_e) + \sum_{e \neq j} \text{tr}(\Delta_e^\top H_{ej} \Delta_j). \quad (8)$$

Expert-local or affinity-weighted GPTQ minimizes only

$$\mathcal{L}_{\text{diag}}(\mathbf{Q}) = \sum_e \text{tr}(\Delta_e^\top H_{ee} \Delta_e), \quad (9)$$

possibly with a different estimate of each diagonal block. It cannot select quantization errors that cancel after expert aggregation because the second sum in (8) is absent.

Proposition 2 (Separation condition). *If $H_{ej} = 0$ for all $e \neq j$, the joint problem separates exactly into E independent expert problems. Conversely, if the feasible error differences span the corresponding coordinates, exact separation for all weights requires zero cross blocks.*

Remark 1. The proposition is also a falsification criterion. If measured cross blocks are negligible, or if a full-joint oracle gives no improvement, the proposed paper direction should stop rather than add a more elaborate solver.

4 What can actually be guaranteed

4.1 Global-model dominance

Let

$$\mathbf{Q}_{\text{ind}} \in \mathcal{Q} \quad (10)$$

be any result produced by independent RTN, GPTQ, or affinity-weighted GPTQ, and define

$$\mathbf{Q}_{\text{joint}}^* \in \arg \min_{\mathbf{Q} \in \mathcal{Q}} \mathcal{L}_{\text{joint}}(\mathbf{Q}). \quad (11)$$

Theorem 1 (Joint optimum dominates every independent solution). *Under the same fixed quantization lattice,*

$$\mathcal{L}_{\text{joint}}(\mathbf{Q}_{\text{joint}}^*) \leq \mathcal{L}_{\text{joint}}(\mathbf{Q}_{\text{ind}}). \quad (12)$$

Proof. $\mathbf{Q}_{\text{ind}}$ is feasible for the joint problem because the feasible set is the same product lattice. A global minimum over that set cannot have a larger objective than a feasible point. $\square$

This theorem validates the *model*, not a greedy implementation. Full-joint GPTQ can still be worse than an independent solution because GPTQ is an ordered heuristic for a discrete problem. The toy study therefore includes exhaustive enumeration and a baseline-preserving solver.

4.2 Finite monotone joint correction

Fix one output column. Index the $P = Ed$ stacked input coordinates by i , write $\epsilon_i = w_i - q_i$, and hold every code except q_i fixed. The terms depending on q_i are

$$f_i(q_i) = H_{ii}(w_i - q_i)^2 + 2(w_i - q_i)c_i, \quad c_i = \sum_{j \neq i} H_{ij}\epsilon_j. \quad (13)$$

Because q_i lies in a small finite codebook, its exact coordinate minimizer is obtained by enumerating the levels.

Theorem 2 (Monotone baseline-preserving correction). *Initialize $\mathbf{Q}^{(0)} = \mathbf{Q}_{\text{ind}}$. Repeatedly replace one code by the minimizer of (13). Then*

$$\mathcal{L}_{\text{joint}}(\mathbf{Q}^{(t+1)}) \leq \mathcal{L}_{\text{joint}}(\mathbf{Q}^{(t)}) \leq \mathcal{L}_{\text{joint}}(\mathbf{Q}_{\text{ind}}). \quad (14)$$

With deterministic tie handling, the method terminates after finitely many strict decreases at a coordinate-wise discrete minimum.

Proof. Each update exactly minimizes the full objective over a subset containing the current code, so it cannot increase the objective. The Cartesian product codebook is finite; therefore an infinite sequence of distinct strict improvements is impossible. $\square$

The theorem gives the desired first-stage correction guarantee without QAT, reinforcement learning, or a suffix Jacobian. It does not claim a globally optimal code assignment.

For M output columns, $P = Ed$ coordinates, and S sweeps, the direct implementation costs $O(SP^2M)$ and stores $O(P^2)$ Hessian entries. This is intentionally an oracle implementation, not a production claim.

5 From block error to later routing

Reducing (5) is the correct local objective, but it is not an unconditional terminal-reward theorem.

Algorithm 1 Full-Hessian MoEJointQuant verification solver**Require:** FP weights $\mathbf{W}$, calibration $(\mathbf{a}_{ne}, g_{ne})$, fixed quantization lattice $\mathcal{Q}$

- 1: Construct Φ and the dense $\mathbf{H} = N^{-1}\Phi^\top\Phi$
- 2: Quantize every expert independently to obtain $\mathbf{Q}^{(0)}$
- 3: **for** $t = 0, 1, \dots$ until no improvement **do**
- 4: **for** coordinate i in descending H_{ii} order **do**
- 5: Compute $c_i = \sum_{j \neq i} H_{ij}(w_j - q_j)$
- 6: Set $q_i \leftarrow \arg \min_{q \in \mathcal{Q}_i} H_{ii}(w_i - q)^2 + 2(w_i - q)c_i$
- 7: **end for**
- 8: **end for**
- 9: **return** $\mathbf{Q}$ and the complete monotone objective history

Proposition 3 (Fixed-route suffix bound). *Suppose the suffix from the current block output to terminal logits is locally L -Lipschitz on the states under consideration and does not cross a routing discontinuity. Then a block perturbation $\delta\mathbf{y}$ satisfies*

$$\|z(\mathbf{y} + \delta\mathbf{y}) - z(\mathbf{y})\|_2 \leq L \|\delta\mathbf{y}\|_2. \quad (15)$$

Thus lower block MSE tightens a valid upper bound, although the unknown L may be large.

For the next router $r(\mathbf{h}) = R\mathbf{h}$, define the FP Top- k logit margin

$$\mu(\mathbf{h}) = \min_{e \in \text{TopK}(r)} r_e - \max_{j \notin \text{TopK}(r)} r_j. \quad (16)$$

Proposition 4 (Sufficient Top- k stability condition). *The next Top- k set is unchanged whenever*

$$2\|R\delta\mathbf{h}\|_\infty < \mu(\mathbf{h}). \quad (17)$$

Proof. Every selected logit can decrease by at most $\|R\delta\mathbf{h}\|_\infty$, while every unselected logit can increase by the same amount. Condition (17) preserves their ordering across the Top- k boundary. $\square$

This explains the intended role of block correction: reduce the perturbation that drives route changes, not force every quantized trajectory to equal the FP trajectory. Tokens with a small router margin remain inherently fragile.

6 Calibration as a separate generalization problem

Let H_{cal} and H_{test} denote the augmented-feature second moments. For a fixed quantization error $\Delta = \mathbf{W} - \mathbf{Q}$,

Proposition 5 (Covariance-shift bound).

$$|\mathcal{L}_{\text{test}}(\mathbf{Q}) - \mathcal{L}_{\text{cal}}(\mathbf{Q})| \leq \|H_{\text{test}} - H_{\text{cal}}\|_2 \|\Delta\|_F^2. \quad (18)$$

Proof. Use $|\text{tr}(\Delta^\top A \Delta)| \leq \|A\|_2 \|\Delta\|_F^2$ with $A = H_{\text{test}} - H_{\text{cal}}$. $\square$

Equation (18) identifies the real calibration target: cover the second moment of gate-weighted expert activations, including the off-diagonal co-routing blocks. Merely equalizing marginal expert counts does not guarantee this.

The executable toy study compares four selectors:

- **Random:** samples the naturally skewed candidate pool.
- **Task-balanced oracle:** uses synthetic task labels and is not available in real PTQ.
- **Route-balanced:** stratifies by the unordered Top- k expert signature; it needs no task labels.
- **Feature pivot:** greedily spans the augmented feature matrix, a fast approximation to D-optimal design.

A future real-model selector should combine route-signature coverage with a feature-space criterion. Full token-by-token trajectory balancing is unnecessary at this stage: the object that appears in the theorem is $\Phi^\top \Phi$, not an exponentially long route sequence.

7 Executable toy MoE experiment

7.1 Design

The reference implementation is `moejointquant.py` in the repository root. Its default experiment uses:

- five experts, Top-2 routing, $d_{\text{model}} = 12$, and $d_{\text{ff}} = 7$;
- ten synthetic tasks, one for every expert pair, with a deliberately skewed task mixture;
- 2-bit fixed per-expert, per-output weight grids;
- 768 candidate calibration samples, 96 selected samples, and 1,024 test samples;
- five model/data seeds;
- an exact dense 35×35 co-routing Hessian, with no structural approximation.

The evaluated methods are RTN, expert-local GPTQ, affinity-weighted expert GPTQ, dense joint GPTQ, and the monotone joint coordinate solver initialized from affinity GPTQ. Besides block NMSE, the experiment propagates the output through a fixed nonlinear suffix, measures teacher-label agreement, and compares the next Top-2 routing set. These are diagnostics, not training objectives.

7.2 Tiny exact oracle

For three experts and two hidden coordinates per expert, all 2-bit assignments can be enumerated. Across five seeds:

Metric	Mean
Exact joint gain over exact independent	15.11%
Coordinate-solver gain over exact independent	13.28%
Exact dominance checks passed	5 / 5
Coordinate baseline guarantees passed	5 / 5

This is the cleanest evidence for the modeling claim: the exact joint lattice solution is better before any acceleration or calibration-selection method is added. The remaining 1.83 percentage-point gap is solver error, not a failure of the objective.

7.3 Held-out results

Table 1 uses random calibration and evaluates on a held-out set from the same skewed mixture. Joint coordinate correction reduces block NMSE by 17.96% relative to affinity GPTQ and also improves the nonlinear suffix.

Table 1: Five-seed mean with random calibration and skewed IID test. Lower is better except agreement and route overlap.

Method	Block NMSE	Suffix NMSE	Agreement	Route overlap
RTN	0.2584	0.0659	0.8461	0.9300
Independent GPTQ	0.2084	0.0581	0.8471	0.9235
Affinity GPTQ	0.2031	0.0592	0.8488	0.9252
Joint GPTQ	0.1775	0.0478	0.8629	0.9364
Joint coordinate	0.1666	0.0461	0.8648	0.9345

Random calibration does not cover a balanced test distribution: its joint-coordinate block NMSE rises to 0.2867 and RTN reaches 0.2181. This negative result is important. It shows that a lower training quadratic is not enough under route-covariance shift, exactly as Proposition 5 predicts.

Table 2 changes only the calibration selector to route-balanced and evaluates all expert-pair tasks uniformly. Joint coordinate correction is 17.57% lower than affinity GPTQ and 35.01% lower than RTN in block NMSE.

Table 2: Five-seed mean with route-balanced calibration and balanced test.

Method	Block NMSE	Suffix NMSE	Agreement	Route overlap
RTN	0.2181	0.0664	0.8449	0.9263
Independent GPTQ	0.1640	0.0586	0.8531	0.9251
Affinity GPTQ	0.1720	0.0601	0.8547	0.9229
Joint GPTQ	0.1585	0.0545	0.8533	0.9235
Joint coordinate	0.1417	0.0537	0.8578	0.9244

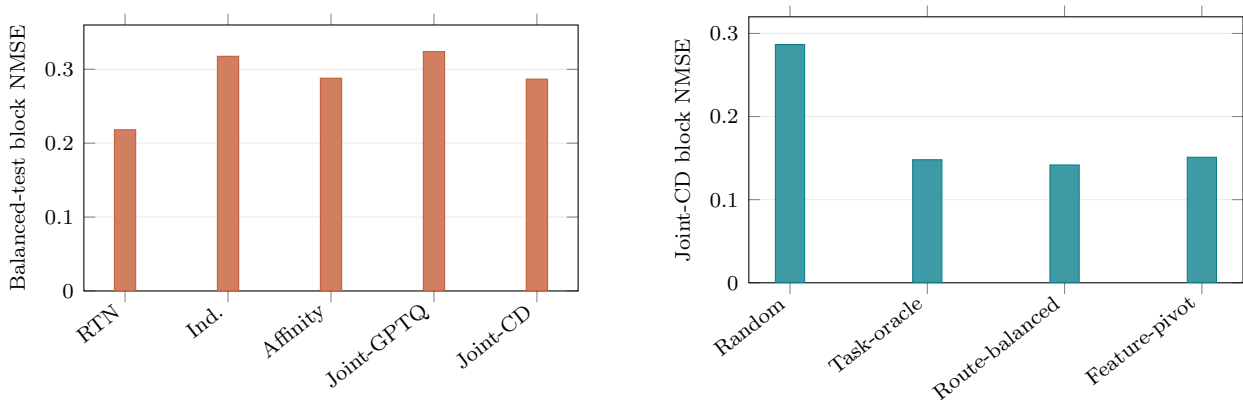


Figure 1: Left: balanced-test method comparison under random calibration, deliberately exposing distribution shift. Right: calibration-selector comparison for the fixed joint-coordinate method.

7.4 Interpretation

The toy evidence supports three statements and no more:

1. Cross-expert terms create a real discrete optimization gain in a controlled MoE.
2. A solver initialized from an independent result can realize most of the tiny exact-oracle gain while preserving a calibration no-worse guarantee.
3. Held-out transfer requires calibration coverage of co-routing structure; random perplexity-style samples are not automatically sufficient.

It does not establish a gain on a pretrained language model, activation quantization, all expert projections, or autoregressive task success.

8 Most feasible real-model verification

The next experiment should remain small and diagnostic.

1. Select one open MoE with roughly 1B active parameters and expose one or a few MoE layers. Quantize only expert down projections first, leaving attention, routers, shared experts, and the suffix in FP16.
2. Cache 128–512 sequences but cap the number of gate-weighted activation rows per layer. Run both random and route-signature-stratified subsets with identical token budgets.
3. On a small channel slice, compute an exhaustive or beam-search joint oracle. On the full selected layer, run the dense joint coordinate solver. Do not add grouping or low-rank Hessian approximations.
4. Compare RTN, independent GPTQ, affinity GPTQ, dense joint GPTQ, and baseline-initialized joint correction. Report calibration and held-out aggregated block error, logits KL after the FP16 suffix, PPL, route-margin buckets, and the next-router Top- k overlap.
5. Repeat over at least three layers and five calibration seeds. Separate tokens by co-routing edge frequency and router margin; an average alone can hide the mechanism.

Go / No-Go criterion. Continue to Hessian acceleration only if (i) the small-slice exact joint oracle consistently improves over the exact independent solution, and (ii) dense baseline-initialized joint correction transfers a material fraction of that gain to held-out aggregated block error and at least one suffix metric. If either condition fails, route correction and specialized calibration data should not be added to the core method.

9 Deferred acceleration

If the dense verifier passes, the full Hessian can be approximated in a second paper stage. Candidate structures include:

- channel slices with complete cross-expert blocks inside each slice;
- expert groups induced by the co-routing graph;
- a block-sparse local term plus a low-rank global residual;

- Woodbury or iterative solves without materializing the full inverse.

Sparse-plus-low-rank decomposition and ADMM are relevant here, but not to the first-stage proof. Introducing them earlier would confound modeling gain with approximation error. The appropriate second-stage comparison is a Pareto curve against the dense oracle: Hessian storage, solve time, and retained joint-objective gain.

10 Reproducibility

The implementation requires PyTorch; Matplotlib is optional for standalone PNG summaries. From the repository root:

```
.venv/bin/python moejointquant.py --self-test
.venv/bin/python moejointquant.py \
  --output hsvdquant/toy/results/moejointquant \
  --seeds 0 1 2 3 4
```

The output directory always contains `results.json`, `metrics.csv`, and a Markdown report; PNG summaries are added when Matplotlib is available. The JSON records all configuration values, every per-seed metric, exact-oracle checks, coordinate histories, and runtime. The plots in this note are rendered directly from those reported means.

11 Conclusion

The most defensible core is not “trajectory correction” and not “a faster MoE Hessian.” It is the exact observation that co-routed experts form one weighted operator, together with a baseline-preserving discrete correction on that operator. The global joint model has a simple dominance theorem; the executable solver has a monotonic calibration guarantee; and the suffix and router analyses state the additional conditions needed for local error to propagate safely. The toy study is positive but also shows why calibration must be evaluated separately. This provides a clean sequence for the paper: prove and verify joint gain first, then approximate its structure, and only then study route-aware calibration selection.

References

- [1] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *ICLR*, 2023.
- [2] Z. Chen et al. MoEQuant: Enhancing Quantization for Mixture-of-Experts Large Language Models via Expert-Balanced Sampling and Affinity Guidance. *ICML*, 2025.
- [3] X. Yuan and J. Yang. Sparse and Low-Rank Matrix Decomposition via Alternating Direction Methods. 2011.